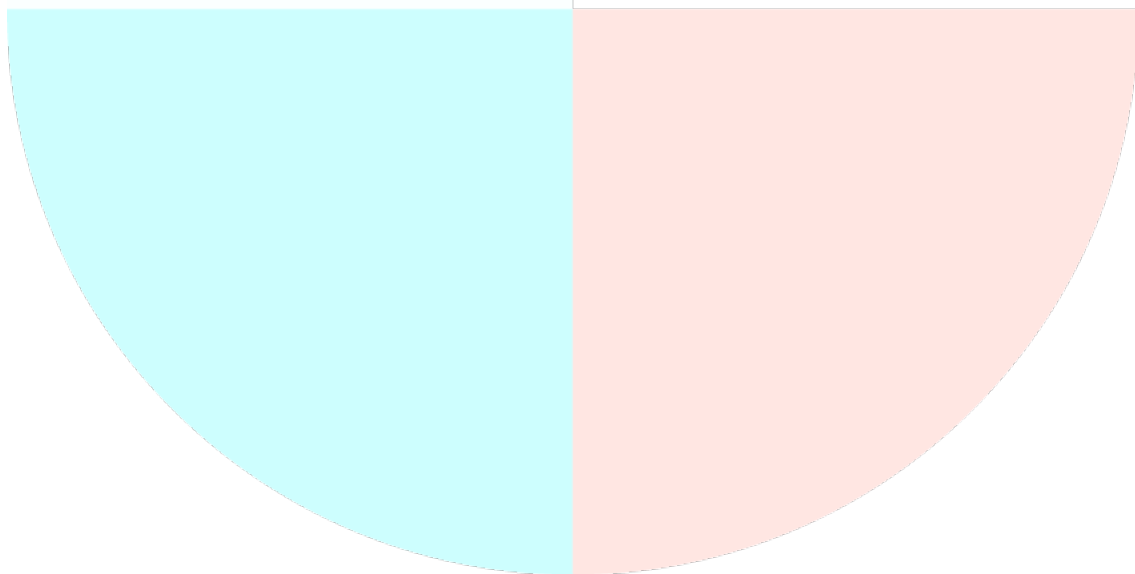


## Bloque 2 - Tema 5

---

SISTEMAS DE GESTIÓN DE BASES DE DATOS  
RELACIONALES, ORIENTADOS A OBJETOS Y  
NOSQL: CARACTERÍSTICAS Y COMPONENTES

---



**PREPARACIÓN OPOSICIONES**  
**TÉCNICOS AUXILIARES DE INFORMÁTICA**

# ÍNDICE

|                                                       |          |
|-------------------------------------------------------|----------|
| <b>ÍNDICE .....</b>                                   | <b>2</b> |
| 1. SISTEMAS DE GESTIÓN DE BASES DE DATOS (SGDB) ..... | 3        |
| 2. SGBD RELACIONALES.....                             | 9        |
| 1. Oracle.....                                        | 11       |
| 2. MySQL.....                                         | 14       |
| 3. MariaDB.....                                       | 15       |
| 4. SQLite.....                                        | 16       |
| 5. PostgreSQL.....                                    | 16       |
| 6. SQL Server.....                                    | 17       |
| 7. Apache Derby.....                                  | 18       |
| 8. MaxDB.....                                         | 18       |
| 9. HSQLDB.....                                        | 18       |
| 10. Aurora.....                                       | 18       |
| 3. SGBD ORIENTADO A OBJETOS.....                      | 19       |
| 4. SGBD NOSQL.....                                    | 21       |
| 1. Cassandra (clave-valor).....                       | 26       |
| 2. Redis (clave-valor).....                           | 26       |
| 3. DynamoDB (clave-valor).....                        | 27       |
| 4. MongoDB (orientada a documentos).....              | 27       |
| 5. CouchDB (orientada a documentos).....              | 27       |
| 6. Neo4j (orientada a grafos).....                    | 28       |
| 7. Bigtable (orientada a columnas).....               | 28       |
| 8. HBase (orientada a columnas).....                  | 28       |
| 9. IndexedDB.....                                     | 28       |
| 5. OTROS TIPOS DE BASES DE DATOS .....                | 30       |

# 1. SISTEMAS DE GESTIÓN DE BASES DE DATOS (SGDB)

El **SGBD** es un conjunto coordinado de programas, procedimientos y lenguajes, que suministra, tanto a los usuarios informáticos, como no informáticos y al administrador, los medios necesarios para describir, recuperar y manipular los datos integrados en la base de datos, asegurando su confidencialidad y seguridad.

## Funciones

Las **funciones** principales de un SGBD son:

- Permitir la **descripción unificada de los datos** que integran la base de datos, su estructura y las relaciones que existen entre ellos, así como la definición de vistas parciales de los datos para distintos usuarios; ofreciendo **flexibilidad** en la definición del modelo. Esta función es llevada a cabo por medio del Lenguaje de Definición de Datos (DDL).
- Facilitar la manipulación de los datos, es decir, el **manejo de grandes volúmenes de datos**, permitiendo a los usuarios de la base de datos añadir, buscar, suprimir o modificar los datos de la misma. Se lleva a cabo por medio del Lenguaje de Manipulación de Datos (DML).
- Proporcionar un **catálogo** (diccionario de datos).
- Garantizar la **independencia física y lógica** de las aplicaciones/usuarios respecto a la representación física de los datos.
- Garantizar la **seguridad y confidencialidad** de la base de datos, mediante el control de acceso de los usuarios a la base de datos y la limitación del acceso a los datos gracias a un sistema de derechos o privilegios, a través del Lenguaje de Control de Datos (DCL).
- Garantizar la **integridad (consistencia)** de los datos. Por una parte, asegura que los datos se almacenan correctamente y, por otra, que las actualizaciones de los usuarios sobre la BD se ejecutan correctamente y que se hacen permanentes.
- Garantizar el **acceso concurrente** a los datos. Permitir que varios usuarios puedan utilizar los mismos datos sin que se produzcan problemas en los accesos simultáneos.
- Proporcionar mecanismos para la **recuperación** ante fallos.
- Ofrecer **escalabilidad**.
- Facilitar estadísticas al vuelo de la explotación de la base de datos, donde se pueda observar su rendimiento.

## Arquitectura

Un SGBD sigue la arquitectura ANSI/X3/SPARC de 3 niveles ya vista en temas anteriores.

### Componentes

- **Preprocesador de consultas:** transforma las consultas en un conjunto de instrucciones de bajo nivel que se dirigen al gestor de la base de datos.
- **Gestor de la base de datos:** interfaz con los programas de aplicación y las consultas de los usuarios. Acepta consultas y examina los esquemas externo y conceptual para determinar qué registro se requieren para satisfacer la petición. Entonces el gestor realiza una llamada al gestor de ficheros para ejecutar la petición. El gestor de la BD se compone de:
  - o Control de autorización: comprueba que el usuario tiene permisos necesarios para llevar a cabo la operación que necesita.
  - o Procesador de comandos: comprobados los permisos del usuario, se pasas el control a este módulo.
  - o Control de la integridad: cuando una operación cambia los datos de la BD, este módulo debe comprobar que la operación a realizar satisface todas las restricciones de integridad necesarias.
  - o Optimizador de consultas: se encarga de determinar la estrategia óptima para la ejecución de las consultas.
  - o Gestor de transacciones: se encarga del procesamiento de las transacciones.
  - o Planificador (scheduler): este módulo es el responsable de asegurar que las operaciones que se realizan concurrentemente sobre la BD tienen lugar sin conflictos.
  - o Gestor de recuperación: garantiza que la BD permanece en un estado consistente en caso de que se produzca algún fallo.
  - o Gestor de buffers (gestor de datos): responsable de transferir los datos entre memoria principal y los dispositivos de almacenamiento secundario.
- **Gestor de ficheros:** maneja los ficheros en disco en donde se almacena la base de datos. Establece y mantiene la lista de estructuras e índices definidos en el esquema interno.
- **Preprocesador o precompilador DML:** convierte las sentencias del DML embebidas en los programas de aplicación en llamadas a funciones estándar escritas en el lenguaje anfitrión. El preprocesador trabaja con el procesador de consultas para generar el código apropiado.
- **Compilador DML:** traduce las instrucciones del DML en un lenguaje de consultas a un plan de evaluación que consiste en instrucciones de bajo nivel que entiende el motor de evaluación de consultas. Una consulta se puede traducir habitualmente en varios planes de ejecución alternativos que proporcionan el mismo resultado. El compilador DML también realiza optimización de consultas, es decir, elige el plan de evaluación de menor coste de entre todas las alternativas.
- **Intérprete DDL:** convierte las sentencias del DDL en un conjunto de tablas que contienen metadatos. Las definiciones de estas tablas se registran en el diccionario de datos.
- **Motor de ejecución:** ejecuta las instrucciones de bajo nivel generadas por el compilador DML.
- **Gestor del diccionario de datos:** controla los accesos al diccionario de datos y se encarga de mantenerlo.

**Transacciones: integridad en las actualizaciones y acceso concurrente**

Las operaciones de acceso de las aplicaciones a la base de datos ponen en peligro su integridad. Estas operaciones se organizan en transacciones.

Una **transacción** es una secuencia de operaciones de acceso a la BD que constituye una unidad lógica de ejecución.

Como ejemplo de transacción, imaginemos que ordenamos la transferencia por la compra de un producto en Amazon. Este proceso está formado por un conjunto de operaciones:

- 1) Comprobar que nuestra cuenta es válida y está operativa.
- 2) Comprobar si hay saldo suficiente.
- 3) Comprobar los datos de la cuenta del vendedor.
- 4) Retirar el dinero de nuestra cuenta.
- 5) Ingresar el dinero en la cuenta del vendedor.

Podemos observar en el ejemplo que las operaciones de la transacción deben ejecutarse todas o ninguna, que si falla la operación 4 o 5 la BD quedará en un estado inconsistente, que los cambios en los datos no deben ser visibles hasta que la transacción sea confirmada y que una vez finalice con éxito la ejecución de las operaciones se deben confirmar los cambios en la BD.

Del ejemplo visto, se desprenden las propiedades que deben cumplir las transacciones:

- **ATOMICIDAD**: una transacción es una unidad atómica de ejecución. O se ejecutan todas sus operaciones o ninguna.
- **CONSISTENCIA**: la transacción debe dar lugar a un estado de la BD consistente, cumpliéndose todas las restricciones de integridad, es decir, nunca dejará datos inconsistentes.
- **AISLAMIENTO**: las modificaciones introducidas por una transacción no confirmada no son visibles al resto de transacciones.
- **DURABILIDAD**: la confirmación implica la grabación de los cambios introducidos en la BD, de modo que serán permanentes.

A estas propiedades se las suele conocer como **propiedades ACID** por sus siglas en inglés: Atomicity, Consistency, Isolation y Durability.

Por tanto, esta unidad lógica de trabajo es un conjunto de sentencias que se ejecutan como si fuesen una única sentencia. Además, existe una relación entre las sentencias que forman parte de una transacción, de tal forma que la no ejecución de una sentencia supone que carezca de sentido la ejecución de las demás.

Las transacciones se inician de forma implícita al ejecutar sentencias como CREATE, ALTER, INSERT, UPDATE o de forma explícita con la sentencia SET TRANSACTION.

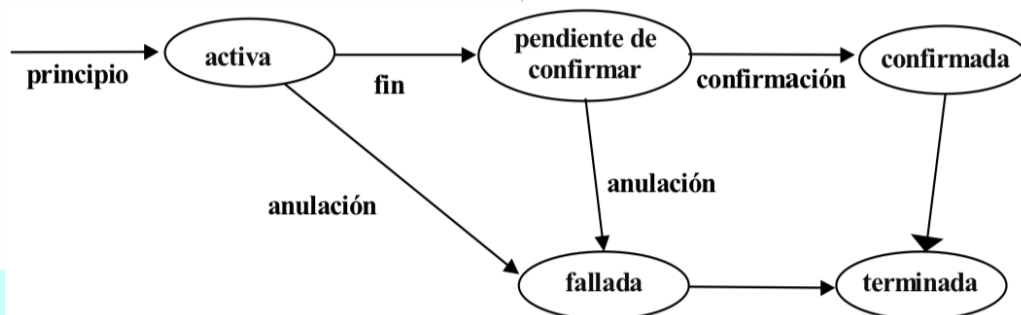
La finalización de una transacción debe ser explícita con una de las siguientes sentencias:

- **COMMIT**: confirma los cambios realizados en la base de datos durante la ejecución de la transacción.

En bases de datos distribuidas se utiliza el algoritmo **protocolo commit en 2 fases** (two phase commit) para la gestión de transacciones. Está basado en la existencia de un coordinador. Al resto de nodos de la red se les llama participantes. El protocolo se divide en dos fases: fase de preparación de commit donde el coordinador intenta preparar a todos los participantes y fase commit donde el coordinador completa las transacciones a todos los participantes.

- **ROLLBACK**: deshace todos los cambios que se hayan producido en la base de datos y se vuelve al estado anterior al inicio de la ejecución de la transacción.
- **SAVEPOINT**: define un punto de recuperación en una transacción para poder revertirla hasta dicho punto mediante ROLLBACK.

Los estados por los que pasa una transacción están representados en el siguiente diagrama:



Por otro lado y en relación con las transacciones, es misión del SGBD la gestión de la concurrencia y los bloqueos. Para ello, supongamos las operaciones de acceso:

- **leer(X)**: consulta de un dato.
- **escribir(X)**: actualización (inserción, modificación o borrado) de un dato.

Ahora bien, supongamos la ejecución de dos transacciones concurrentes. La primera transacción retira 100€ de una cuenta y la segunda transacción retira 200€ de la misma cuenta. Como podemos observar en la imagen siguiente, el resultado es incorrecto.

**Cuentas**

| Nro. | Saldo |
|------|-------|
| 123  | 1000  |
| 555  | 2000  |
|      |       |

**Cuentas**

| Nro. | Saldo |
|------|-------|
| 123  | 800   |
| 555  | 2000  |
|      |       |

| Tiempo | P1                   | P2                   |
|--------|----------------------|----------------------|
| t1     | leer(123, saldo)     |                      |
| t2     |                      | leer(123, saldo)     |
| t3     | saldo←saldo-100      |                      |
| t4     |                      | saldo←saldo-200      |
| t5     | escribir(123, saldo) |                      |
| t6     |                      | escribir(123, saldo) |

Visto en el ejemplo el acceso concurrente a los datos, el SGBD controla los accesos concurrentes para evitar la pérdida de actualizaciones, la obtención de información incoherente y la lectura de datos no confirmados que podrían ser anulados.

Los problemas de interferencias entre transacciones que nos podemos encontrar son:

- **Lectura NO REPETIBLE**: dos transacciones paralelas intentan modificar el mismo objeto de la base de datos, leyendo ambas el valor antes de que la otra transacción lo actualice. Es decir, ocurre cuando una transacción T1 lee dos veces un valor y no coinciden porque una segunda transacción T2 ha modificado dicho valor entre ambas lecturas.

T1 lee fila A.

T2 modifica fila A.

T1 lee de nuevo la fila A y obtiene resultados diferentes.

- **Lectura SUCIA**: cuando se permite que una transacción lea una fila que fue modificada por otra transacción que todavía no hizo commit y, por lo tanto, no ha almacenado el nuevo valor. Es decir, ocurre cuando una transacción T2 lee un dato modificado por otra transacción T1 antes de que haya realizado el commit (viola la propiedad de aislamiento). Si T1 falla o realiza otra modificación del dato, el valor leído por T2 nunca ha llegado a ser válido.

T1 inserta fila A.

T2 lee fila A.

T1 ejecuta rollback (se pierde la fila A).

- **Lectura FANTASMA**: es un caso particular de las lecturas no repetibles. Durante una transacción se ejecutan dos consultas idénticas, y entre ambas consultas otra transacción modifica datos que afectan a esta consulta. La colección de filas retornada por la segunda consulta será distinta a la de la primera. Esto es, ocurre cuando una transacción T1 realiza varias consultas iguales y una segunda transacción T2 inserta datos entre la realización de ambas consultas, provocando que la segunda lectura de T1 muestre más datos que la primera consulta.

T1 lee un conjunto de filas (satisfacen cláusula WHERE consulta SQL).

T2 inserta una nueva fila (satisface cláusula WHERE consulta anterior).

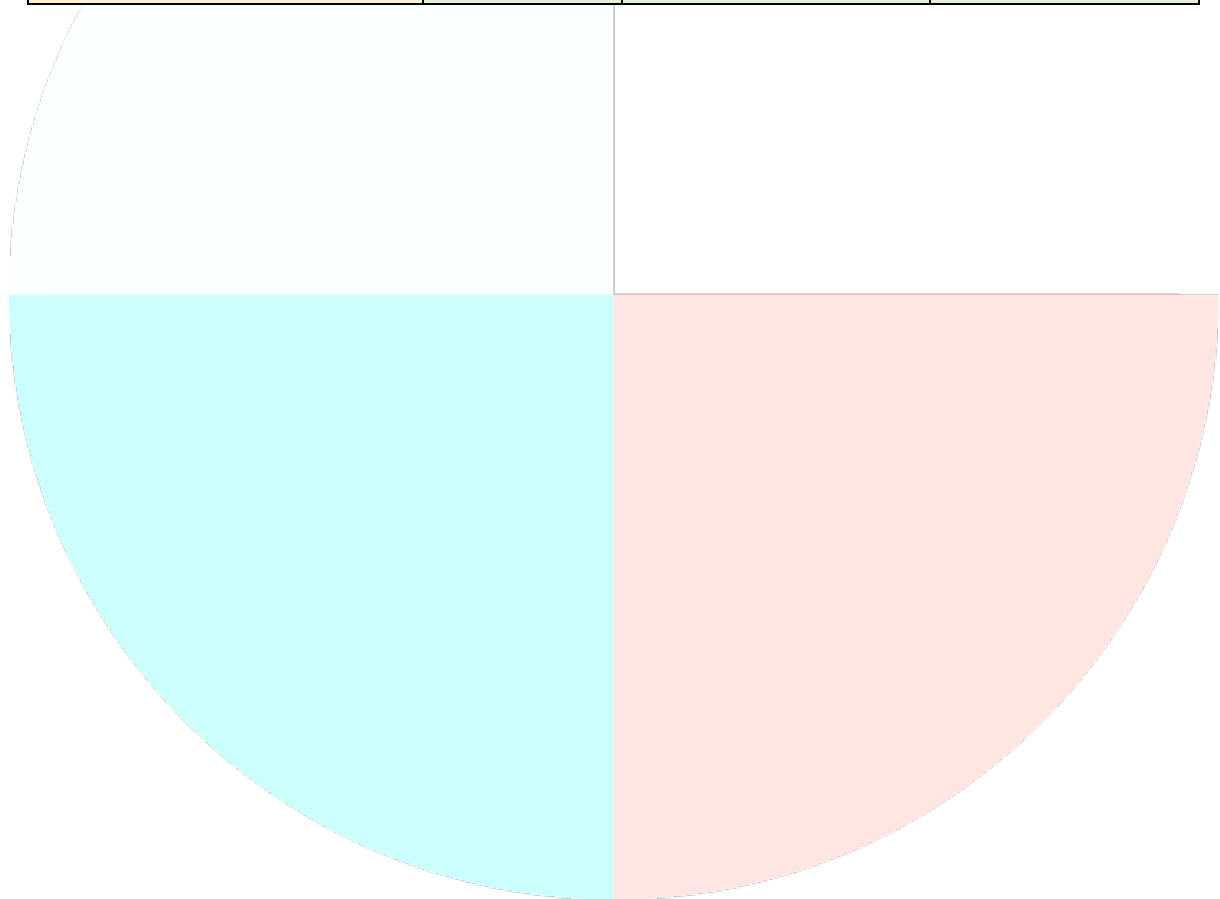
T1 vuelve a ejecutar consulta paso 1 y obtiene una fila más

Según el nivel de error que se considere aceptable en nuestro sistema, se establecerá el **NIVEL DE AISLAMIENTO** necesario para evitar alguno o todos los problemas descritos en el apartado anterior. Los niveles de aislamiento de las transacciones contemplados en ANSI SQL-92 son (de menor a mayor nivel de aislamiento):

- **Lectura NO COMPROMETIDA (read uncommitted)**: los cambios realizados por las transacciones se encuentran disponibles inmediatamente. También denominada lectura no confirmada.
- **Lectura COMPROMETIDA (read committed)**: los cambios realizados por las transacciones sólo se encuentran a disposición del resto cuando se comprometen (se realiza un commit). Previene solo las lecturas sucias. También denominada lectura confirmada.

- **Lectura REPETIBLE (repeatable read):** las filas leídas o actualizadas por una transacción quedan bloqueadas hasta que dicha transacción termina. Previene la lectura sucia y la lectura no repetible.
- **SERIALIZABLE:** las transacciones ejecutadas de manera simultánea producen los mismos efectos que si se ejecutaran en serie. Previene todas las interferencias entre transacciones.

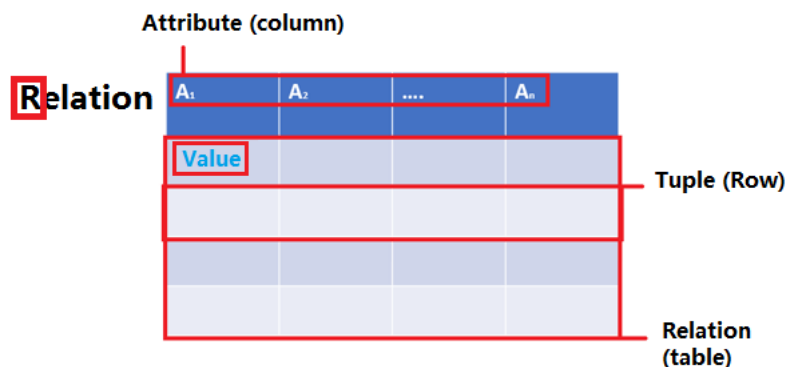
| Nivel de aislamiento    | EVITA la interferencia |                      |                  |
|-------------------------|------------------------|----------------------|------------------|
|                         | Lectura SUCIA          | Lectura NO REPETIBLE | Lectura FANTASMA |
| Lectura NO COMPROMETIDA | NO                     | NO                   | NO               |
| Lectura COMPROMETIDA    | SI                     | NO                   | NO               |
| Lectura REPETIBLE       | SI                     | SI                   | NO               |
| SERIALIZABLE            | SI                     | SI                   | SI               |





## 2. SGBD RELACIONALES

El modelo que goza de **más popularidad** a día de hoy es el relacional, aunque tampoco queda libre de crítica. Su correspondiente sistema de gestión es más conocido como SGBDR (RDBMS en inglés) y como lenguaje utiliza normalmente SQL. Este modelo **basado en tablas**, gira en torno al concepto de relación, un término bien definido en matemáticas y que aquí se utiliza como sinónimo de tabla. Para formular las relaciones se utiliza álgebra relacional, con cuya ayuda puede obtenerse la información de estas relaciones. Este es el principio que fundamenta el lenguaje SQL.



El modelo relacional trabaja con **tablas** independientes que determinan la localización de los datos y sus conexiones. Estos datos conforman un registro (en la imagen, una fila o "tupla") y se guardan en columnas como atributos (en la imagen, de  $A_1$  a  $A_n$ ). La **relación** es lo que resulta de los atributos interrelacionados. Para identificar inequívocamente un registro es elemental la **clave primaria**, que normalmente se define como el primer atributo ( $A_1$ ) y que no puede cambiarse. Dicho de otra manera, esta clave primaria o ID, define la posición exacta del registro con todos los atributos.

### Las 12 reglas de Codd

Las bases de datos relacionales se han ido imponiendo a partir de los años 80 debido a la superioridad que ofrece el modelo de datos en que se basan, el modelo relacional, frente a sus antecesores: el modelo jerárquico o de estructuras en árbol y el modelo Codasyl o de estructuras en red.

En el año 1985, Codd estableció una regla general global, llamada "Regla Cero" y doce principios, de los cuales al menos seis deben satisfacerse para que una base de datos pueda considerarse relacional. Estos son los siguientes:

- **Regla 0. Gestión de una Base de Datos Relacional.** Un sistema de gestión de bases de datos relacionales (SGBDR) debe ser capaz de manejar las bases de datos exclusivamente con sus capacidades relacionales.
- **Regla 1. Representación de la información.** Toda la información en una base de datos relacional se representa explícitamente a nivel lógico y de una manera única, por medio de valores en tablas.

- **Regla 2. Acceso garantizado.** Todos y cada uno de los datos elementales en una base de datos relacional deben ser accesibles lógicamente mediante una combinación de: nombre de tabla, valor de clave primaria y nombre de columna.
- **Regla 3. Representación sistemática de la información que falta.** Los valores nulos deben ser soportados por un SGBD completamente relacional para representar, de modo sistemático, la información desconocida o inaplicable.
- **Regla 4. Catálogo dinámico.** La descripción de la base de datos se representa a nivel lógico en la misma forma que los datos, de forma que los usuarios autorizados puedan aplicar el mismo lenguaje relacional para consultarlo.
- **Regla 5. Sublenguaje global de datos.** Debe existir, al menos, un lenguaje cuyas sentencias sean expresables mediante una sintaxis bien definida, como cadena de caracteres, y capaz de soportar definición de datos, definición de vistas, manipulación de datos, restricciones de integridad, autorizaciones y manejo de transacciones.
- **Regla 6. Actualización de vistas.** Todas las vistas teóricamente actualizables deberán ser también actualizables por el sistema.
- **Regla 7. Inserciones, actualizaciones y eliminaciones de alto nivel.** La capacidad para manejar como un solo operando una relación base o una relación derivada se aplica, no sólo a las recuperaciones de datos, sino también a sus inserciones, actualizaciones y eliminaciones.
- **Regla 8. Independencia física de los datos.** Los programas de aplicaciones y las actividades terminales permanecerán lógicamente inalterados siempre que se realicen cambios en las representaciones de almacenamiento o en los métodos de acceso.
- **Regla 9. Independencia lógica de los datos.** Cuando se efectúe en las tablas cualquier tipo de cambio que preserve la información, los programas de aplicación permanecerán intactos.
- **Regla 10. Independencia de la integridad.** Las reglas de integridad de una base de datos particular deben ser definibles por medio del sublenguaje de datos relacional y almacenables en el catálogo, no en los programas de aplicaciones.
- **Regla 11. Independencia de la distribución.** Un sistema relacional debe tener un sublenguaje de datos que pueda soportar bases de datos distribuidas sin alterar los programas de aplicaciones o actividades finales.
- **Regla 12. Regla de la no inversión (o no subversión).** Si un sistema relacional tiene un lenguaje de bajo nivel, éste no puede ser utilizado para pasar por alto las reglas de integridad y las restricciones expresadas por medio del lenguaje relacional de más alto nivel.

En definitiva, de las reglas de Codd podemos concluir que un sistema de base de datos relacional se caracteriza:

- Por presentar externamente sus datos como tablas, aunque internamente se sigan manejando de forma convencional mediante índices, páginas, etc.
- Por la disponibilidad de un lenguaje para operar con las tablas, que al menos tenga las funciones de recuperación, modificación, selección de subconjuntos de tablas sin

predefinición de caminos de acceso, definición de datos, etc., y que proporcione medios para controlar la integridad, seguridad y consistencia de los datos.

- Por disponer de interfaces que permitan el acceso concurrente desde terminales interactivos y programas de aplicación, así como herramientas estándar para controlar la operación y facilitar los procesos de respaldo y recuperación.

A continuación, se enumeran los SGBD relacionales más destacados:

- Oracle.
- MySQL.
- MariaDB.
- SQLite.
- PostgreSQL.
- SQL Server.
- Apache Derby.
- MaxDB.
- HSQLDB.
- Aurora.

## 1. Oracle

Tradicionalmente, Oracle ha sido el **SGBD relacional por excelencia para el mundo empresarial**, considerado siempre como el más **completo y robusto**, destacando por:



- Soporte de transacciones.
- Estabilidad.
- Escalabilidad.
- Multiplataforma.

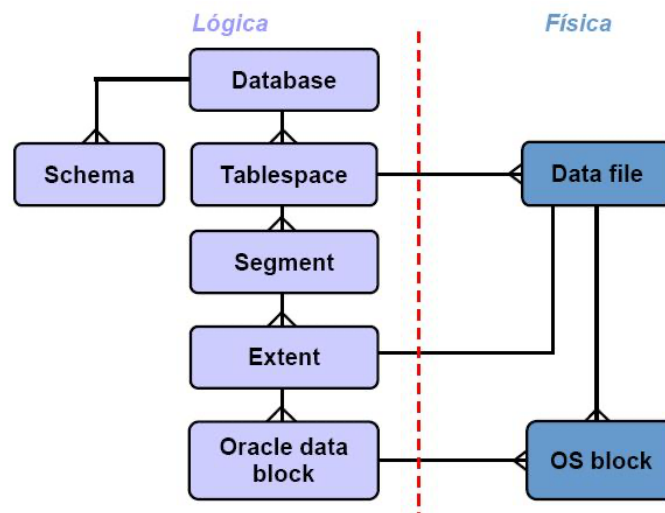
La principal **desventaja**, al igual que SQL Server, es el coste del software ya que, aunque cuenta con una versión gratuita (Express Edition o XE), sus principales opciones son de pago.

**PL/SQL** (Procedural Language/SQL) es el lenguaje de procedimiento cuya sintaxis permite insertar sentencias SQL y se almacena compilado dentro de la base de datos.

Un SGBD Oracle está formado por dos elementos:

- **La instancia de Oracle.** Compuesta por el conjunto de procesos y las estructuras de datos en memoria que requiere el servidor cuando está en funcionamiento para acceder a los ficheros de la base de datos.
- **Archivos de la base de datos.** Los archivos en disco que almacenan de forma permanente la información de la base de datos.

Una BD Oracle tiene una estructura física y una estructura lógica que se mantienen separadamente.



La **estructura FÍSICA** se corresponde con los ficheros del sistema operativo existiendo varios tipos de ficheros:

- **Ficheros de datos (DATAFILES)**: una BD tiene uno o más ficheros de datos, lugar donde se almacena los datos de la base de datos.
- **Ficheros de REDO LOG**: sirven para registrar todos los cambios (inserciones, actualizaciones y borrado de datos y creación, alteración y eliminación de objetos) sobre la BD y poder recuperarla en caso de error.
- **Ficheros de control (CONTROLFILES)**: almacenan información de la estructura física de la BD.

La **estructura LÓGICA** está formada por los **TABLESPACES** y los **OBJETOS** de un esquema de BD: tablas, vistas, índices, etc.

De entre los objetos destacamos las **vistas** y las **vistas materializadas**. En Oracle, una **vista** es un objeto que almacena una consulta y cada vez que se accede a la vista, la consulta se ejecuta y devuelve la información que en ese momento exista en el origen. De otra parte, una **vista materializada** es otro tipo de objeto de Oracle conceptualmente semejante a una vista con la diferencia que almacena el resultado de ejecutar una consulta en un determinado momento, de forma que cada vez que se consulte la vista materializada lo que se recupera es lo que había en el origen en el momento en el que se creó o se refrescó dicha vista materializada.

Cada BD está formada por uno o más tablespaces y cada tablespace pertenece solo a una base de datos. Un **TABLESPACE** es una división lógica de la base de datos.

Por defecto, existe al menos el **tablespace SYSTEM** que almacena el catálogo del sistema (diccionario de datos). Así cuando se crea un objeto se almacena en SYSTEM, salvo que se indique un tablespace específico. Sin embargo, se aconseja reservar el tablespace SYSTEM para uso del servidor y asignar los objetos a otros tablespaces.

Cada tablespace se corresponde con uno o más ficheros de datos y un fichero de datos pertenece solo a un tablespace concreto.

Hay varias razones que justifican este modo de organización de las tablas en espacios de tablas:

- Un tablespace puede quedarse offline debido a un fallo de disco, permitiendo que el SGBD continúe funcionando con el resto.
- Permiten distribuir a nivel lógico/físico los distintos objetos de las aplicaciones.
- Son una unidad lógica de almacenamiento, pueden usarse para aislar completamente los datos de diferentes aplicaciones.
- Oracle permite realizar operaciones de backup/recovery a nivel de tablespace mientras la BD sigue funcionando.

Dentro de cada tablespace se pueden almacenar objetos de distinta naturaleza: tablas, índices, etc. Pero no se pueden mezclar sin más. Necesitamos una manera de separarlos y por este motivo surgen los segmentos.

Un **SEGMENTO** pertenece a un tablespace y está constituido por un conjunto de extensiones, que no son más que grupos de bloques de disco Oracle contiguos. Cuando se borra un segmento, el espacio es devuelto al espacio de tabla.

Un tablespace está formado por varios segmentos. Todos los datos de la BD están almacenados en segmentos. Y existen 5 tipos de segmentos:

- **De datos:** almacenan las tablas.
- **De índices:** permiten un acceso rápido a los datos dependiendo de la cantidad de los mismos (árboles B). Las consultas que sólo referencian a columnas indexadas se resuelven en el índice. Establecen un control de unicidad. Cada índice ocupa un segmento independiente del segmento de datos.
- **De rollback:** son objetos internos de la BD que permiten efectuar la restauración de las transacciones no validadas asegurando la consistencia en lectura. Son reutilizables de manera cíclica. La estructura de los registros de *rollback* es:
  - Identificador de la transacción.
  - Dirección del bloque donde está la tabla.
  - Número de fila.
  - Número de columna.
  - Valor del dato antiguo (antes de ser modificado).
- **Temporales:** son creados por Oracle para un uso temporal cuando debe realizar una ordenación que no le cabe en memoria, y en las operaciones CREATE INDEX, ORDER BY, GROUP BY, DISTINCT, UNION, INTERSECT y MINUS. Son eliminados cuando la sentencia finaliza.

- **De bootstrap:** se crea en SYSTEM y contiene definiciones del diccionario para sus tablas, que se cargan al abrir la BD.

El siguiente nivel de la estructura lógica es la extensión. Una **EXTENSIÓN** es un número específico de bloques de datos Oracle contiguos en disco.

Por último, el nivel más pequeño de granularidad es el de **BLOQUE DE DATOS ORACLE**, que se corresponde con un múltiplo de bloques de disco del sistema operativo.

Respecto a los **tipos de datos** soportados por Oracle tenemos:

- String: CHAR, NCHAR, NVARCHAR2, VARCHAR2, LONG, RAW, LONG RAW.
- Numéricos: NUMBER, BINARY\_FLOAT, BINARY\_DOUBLE.
- Fecha/hora: DATE, TIMESTAMP, TIMESTAMP WITH TIME ZONE, TIMESTATM WITH LOCAL TIME ZONE, INTERVAL.
- Objetos: BFILE, BLOB, CLOB, NCLOB.

## 2. MySQL

Es el sistema gestor de bases de datos relacional por excelencia.



Es un SGBD **multihilo y multiusuario** utilizado en la gran parte de las páginas web actuales. Además, es el más usado en aplicaciones creadas como software libre.

Se ofrece bajo la GNU GPL aunque también es posible adquirir una licencia para empresas que quieran incorporarlo en productos privativos. Resaltar que desde la compra por parte de Oracle se está orientando a este ámbito empresarial.

Las principales ventajas de este Sistema Gestor de Bases de datos son:

- Facilidad de uso y gran rendimiento.
- Facilidad para instalar y configurar.
- Soporte multiplataforma.
- Soporte TLS/SSL.

La principal desventaja es la escalabilidad, es decir, no trabaja de manera eficiente con bases de datos muy grandes que superan un determinado tamaño.

MySQL dispone de varios **motores de bases de datos** (subsistema que permite al SGBD la creación, lectura, actualización y eliminación de datos), destacando:

- **InnoDB:** subsistema de almacenamiento estándar. Ofrece accesos de lectura y escritura seguros para transacciones. Motor por defecto.
- **MyISAM:** subsistema de almacenamiento de versiones antiguas. Ofrece accesos rápidos a índices y tablas, sin embargo, no ofrece seguridad en las transacciones.

Los tipos de datos soportados son:

- String: CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT, ENUM, SET, BINARY, VARBINARY, JSON.
- Numérico: BIT, TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT, DECIMAL, FLOAT, DOUBLE, BOOLEAN.
- Fecha/hora: DATE, DATETIME, TIMESTAMP, TIME, YEAR.
- Objetos: TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB.

En la arquitectura de aplicaciones LAMP (Linux+Apache+MySQL/MariaDB+PHP) este SGBD es uno de sus componentes.

Documentación técnica online: <https://dev.mysql.com/doc/refman/8.0/en/>

### 3. MariaDB

Este SGBD relacional se lanzó en 2009 y es una **derivación de MySQL** que cuenta con la mayoría de las características de este e incluye varias extensiones.



Nace a partir de la adquisición de MySQL por parte de Oracle para seguir la filosofía **Open Source** bajo la licencia GNU GPL y tiene la ventaja de que es totalmente compatible con MySQL.

Entre las principales **características** de este Sistema Gestor de Bases de datos se encuentran:

- Aumento de motores de almacenamiento.
- Gran escalabilidad.
- Seguridad y rapidez en transacciones.
- Extensiones y nuevas características relacionadas con su aplicación para Bases de datos NoSQL.

No tiene desventajas muy aparentes salvo algunas pequeñas incompatibilidades en la migración de MariaDB y MySQL o pequeños atrasos en la liberación de versiones estables.

Los motores de bases de datos disponibles son los mismos que MySQL: InnoDB y MyISAM.

Los tipos de datos soportados son:

- String: CHAR, VARCHAR, TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT, ENUM, SET, BINARY, VARBINARY, JSON.
- Numérico: BIT, TINYINT, SMALLINT, MEDIUMINT, INT, BIGINT, DECIMAL, FLOAT, DOUBLE, BOOLEAN.
- Fecha/hora: DATE, DATETIME, TIMESTAMP, TIME, YEAR.
- Objetos: TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB.



En la arquitectura de aplicaciones LAMP (Linux+Apache+MySQL/MariaDB+PHP) este SGBD es uno de sus componentes.

#### 4. SQLite

Es un SGBD relacional contenido en una biblioteca escrita en lenguaje C utilizada en multitud de aplicaciones actuales ya que es **de dominio público** y las consultas son muy eficientes.



Las principales **características** de SQLite son:

- El tamaño, al tratarse de una biblioteca, es mucho menor que cualquier SGBD.
- Soporta las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) logrando gran estabilidad.
- Tamaño máximo base de datos 281 TB (264 Tibit).
- Gran portabilidad y rendimiento.

La gran **desventaja** de SQLite es que no es fácilmente escalable.

#### 5. PostgreSQL

Este sistema gestor de base de datos relacional está **orientado a objetos** y es de **código abierto**, publicado bajo la licencia BSD.



Sus principales **características** son:

- Control de Concurrencias MultiVersión (MVCC), permitiendo que mientras se accede a una tabla en modo escritura es posible acceder en modo lectura sin bloqueos.
- Compatibilidad con varios lenguajes de programación.
- Multiplataforma.
- Incremento de tipos de datos nativos.
- Disponer de la herramienta **pgAdmin** para la administración de las bases de datos de una forma fácil e intuitiva.
- Robustez, eficiencia y estabilidad.
- Soporta JSON.

La principal **desventaja** es la lentitud para la administración de bases de datos pequeñas ya que está optimizado para gestionar grandes volúmenes de datos.

PostgreSQL se basa en el clásico modelo **cliente-servidor**: el componente central que hace de servidor, con el nombre de "**postmaster**", administra todos los archivos de la BD, así como todas las conexiones que se establecen con el servidor de la BD para las funciones de



comunicación (input/output). Para establecer estas conexiones solo se necesita un programa-cliente, si bien el paquete de software de PostgreSQL ya integra de forma nativa el cliente [psql](#) para operarlo desde la terminal de líneas de comando.

Los tipos de datos soportados son:

- String: carácter varying, varchar, character, char, text.
- Numéricos: smallint, integer, bigint, decimal, numeric, real double precisión, smallserial, serial, bigserial.
- Binarios: byte.
- Fecha/hora: timestamp, timestamp with time zone, time, interval.
- Lógicos: boolean.
- Geométricos: point, line, lseg, box, path, polygon, circle.
- Direcciones de red: cidr, inet, macaddr, macaddr8.
- Búsqueda de texto: tsvector, tsquery.
- Otros: JSON, XML, arrays.

Documentación técnica online:

<https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf>

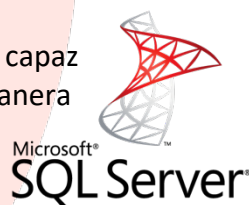
## 6. SQL Server

Es un sistema gestor de bases de datos relacionales basado en el lenguaje **Transact-SQL**, capaz de poner a disposición de muchos usuarios grandes cantidades de datos de manera simultánea.

Es un sistema propietario de **Microsoft**. Sus principales **características** son:

- Soporte exclusivo por parte de Microsoft.
- Escalabilidad, estabilidad y seguridad.
- Posibilidad de cancelar consultas.
- Potente entorno gráfico de administración que permite utilizar comandos DDL y DML.
- Aunque es nativo para Windows puede utilizarse desde hace ya un tiempo en otras plataformas como Linux o Docker.
- Soporte de transacciones.
- Permite obtener instantáneas (snapshot) de la BD.

Su principal **desventaja** es el precio. Cuenta con un plan gratuito (Express) pero lo normal es la elección de alguno de los **planes de pago** disponibles (Standard, Developer, Enterprise o SQL Azure, la versión de SQL Server en la nube).



De forma análoga a Oracle, maneja vistas materializadas, denominándose en este sistema vista indexada.

Conviene resaltar **Machine Learning Services** que es una característica de SQL Server que proporciona la capacidad de ejecutar scripts de Python y R con datos relacionales.

Documentación técnica online: <https://learn.microsoft.com/es-es/sql/sql-server/>.

## 7. Apache Derby

Es un SGBD relacional desarrollado en Java, de código abierto bajo licencia Apache 2.0 License.



## 8. MaxDB

Es un SGBD relacional de la empresa SAP. Originalmente se denominó Adabas.

## 9. HSQLDB

**Hyperthreaded Structured Query Language Database** es un SGBD relacional desarrollado en Java e incluido en OpenOffice para dar soporte a Base. Es multiplataforma y de código abierto bajo licencia BSD.

## 10. Aurora

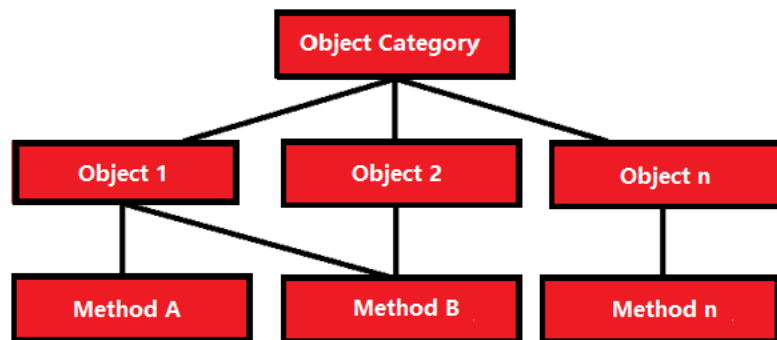
Amazon Aurora es una SGBD relacional compatible con MySQL y PostgreSQL creada para la nube. Combina el rendimiento y la disponibilidad de las bases de datos empresariales tradicionales con la simplicidad y la rentabilidad de las bases de datos de código abierto.

Amazon Aurora es hasta cinco veces más rápida que las bases de datos de MySQL estándar y tres veces más rápida que las bases de datos de PostgreSQL estándar. Ofrece la seguridad, disponibilidad y fiabilidad de las bases de datos de nivel comercial. Amazon Aurora está completamente administrada por Amazon Relational Database Service (RDS), que automatiza las tareas administrativas demandantes como el aprovisionamiento de hardware, la configuración de bases de datos, la aplicación de parches y las copias de seguridad.

Amazon Aurora ofrece un sistema de almacenamiento distribuido, tolerante a errores y de recuperación automática que ajusta su escala automáticamente hasta 128 TiB por instancia de base de datos. Amazon Aurora suministra alto rendimiento y disponibilidad con hasta 15 réplicas de lectura de baja latencia, recuperación a un momento dado y generación de copias de seguridad continua.

### 3. SGBD ORIENTADO A OBJETOS

Las bases de datos de objetos no nacen hasta finales de 1980 y hasta hoy, solo han encontrado una **escasa aplicación**. Estas bases de datos, disponibles también en formato open source, suelen utilizarse en plataformas Java y .NET. La más conocida es db4o, que destaca, sobre todo, por un escaso uso de la memoria. Las bases de datos de objetos acostumbran a trabajar con el **lenguaje OQL**, muy similar a SQL.



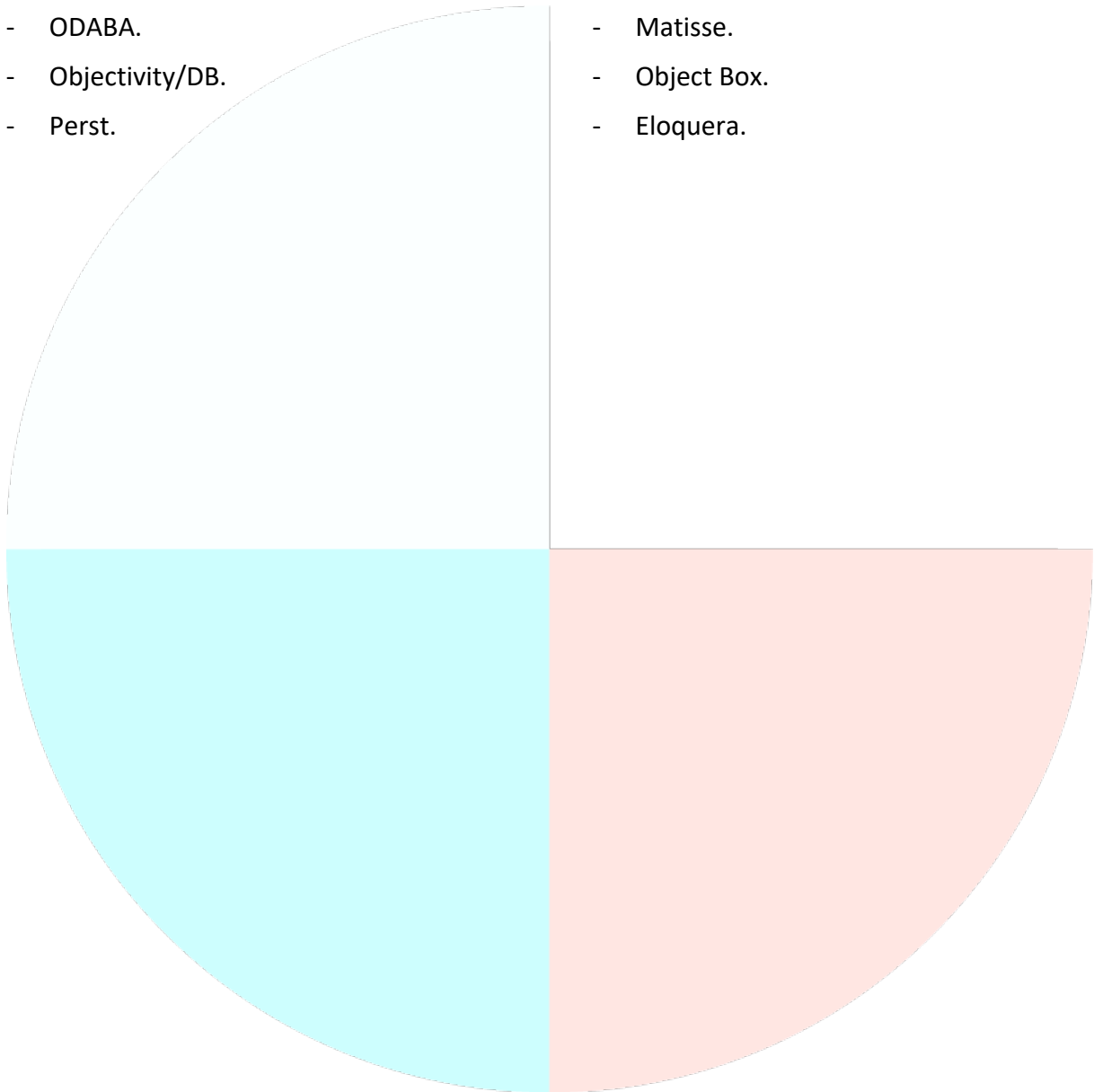
En el modelo orientado a objetos, los datos se guardan en un **objeto** junto con sus funciones (métodos) y los atributos que los describen más en profundidad. En un sistema de gestión de bases de datos de objetos, son los métodos, depositados en el objeto junto con los datos, los que definen cómo se accede al objeto.

Los objetos pueden ser **complejos** y estar compuestos por múltiples tipos de datos, son únicos dentro del sistema de base de datos y se identifican con un **identificador de objeto** (OID en inglés) único. Como puede verse en la figura de arriba, los objetos se agrupan en **clases** (object category), dando como resultado una **jerarquía** de clases. Pese a la aparente similitud con el modelo jerárquico, aquí predomina el paradigma orientado a objetos y no existe ninguna relación padre-hijo fija. Aún así, a través de la clase puede definirse el método para el acceso.

Las ventajas de las bases de datos orientadas a objetos destacan, sobre todo, en problemas con **tipos de datos** complejos. Estas bases de datos trabajan, en su mayor parte, de forma autónoma sin recurrir a la normalización y a la correspondencia de ID, permitiendo así almacenar los objetos nuevos de forma relativamente simple y fluida. Sin embargo, las consultas son mucho más ágiles en un sistema de base de datos relacional. La escasa popularidad de los sistemas orientados a objetos resulta en una insuficiente compatibilidad con muchas de las aplicaciones de base de datos que se usan habitualmente.

Ejemplos de SGBD orientado a objetos son:

- Db4o.
- Zope ObjectDB (ZODB).
- Gemstone.
- ObjectDB.
- ObjectStore.
- ODABA.
- Objectivity/DB.
- Perst.
- Wakanda.
- ObjectDataBase++.
- FastObjects.
- VelocityDB.
- EyeDB.
- Matisse.
- Object Box.
- Eloquera.



## 4. SGBD NOSQL

**NoSQL**, también conocido como Not Only SQL, consiste en una amplia gama de SGBD cuyas principales características son el uso de un lenguaje alternativo al SQL y que guardan **información no estructurada**. Es decir, no existen tablas ni columnas, como en el modelo relacional.

Se puede decir que la aparición del término NoSQL aparece con la llegada de la web 2.0 ya que hasta ese momento sólo subían contenido a la red aquellas empresas que tenían un portal, pero con la llegada de aplicaciones como Facebook, Twitter o YouTube, cualquier usuario podía subir contenido, provocando así un crecimiento exponencial de los datos.

Es en este momento cuando empiezan a aparecer los primeros problemas de la gestión de toda esa información almacenada en bases de datos relacionales. En un principio, para solucionar estos problemas de accesibilidad, las empresas optaron por utilizar un mayor número de máquinas pero pronto se dieron cuenta de que esto no solucionaba el problema, además de ser una solución muy cara. La otra solución era la creación de sistemas pensados para un uso específico que con el paso del tiempo han dado lugar a soluciones robustas, apareciendo así el movimiento NoSQL.

Así pues, una base de datos no relacional (**NoSQL**) es aquella base de datos que se caracteriza porque:

- No requiere de estructuras de datos fijas como tablas.
- No garantiza completamente las características ACID.
- Escala muy bien horizontalmente.

Se utilizan en **entornos distribuidos** que han de estar siempre disponibles y operativos y que gestionan un **importante volumen de datos**.

Las instrucciones NoSQL no garantizan las propiedades ACID debido a la sencillez de sus instrucciones, adaptándose mejor al principio BASE:

- **Basically Available**: prioridad de la disponibilidad de los datos.
- **Soft state**: se prioriza la propagación de datos, delegando el control de inconsistencias a elementos externos.
- **Eventual consistency**: se asume que las inconsistencias temporales progresen a un estado final estable.

Esta forma de almacenar la información ofrece ciertas ventajas sobre los modelos relacionales. Entre las ventajas más significativas podemos destacar:

- **Se ejecutan en máquinas con pocos recursos**. Estos sistemas, a diferencia de los sistemas basados en SQL, no requieren de apenas computación, por lo que se pueden montar en máquinas de un coste más reducido.

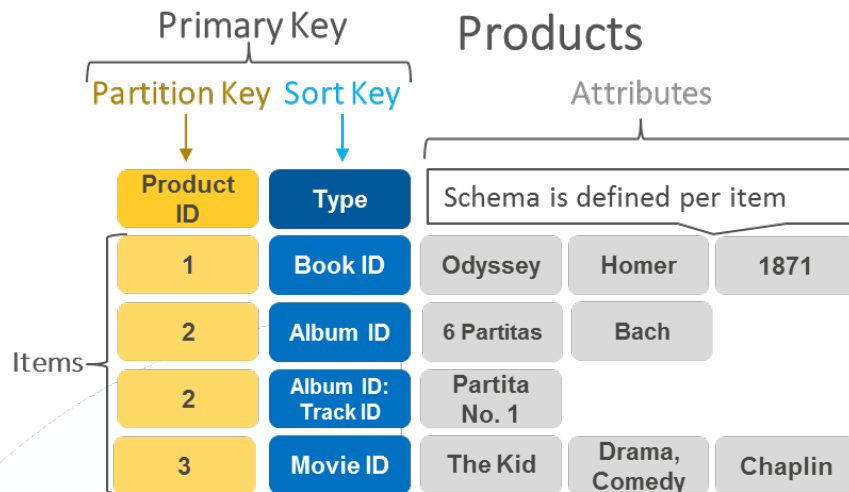
- **Escalabilidad horizontal.** Para mejorar el rendimiento de estos sistemas simplemente se consigue añadiendo más nodos, con la única operación de indicar al sistema cuáles son los nodos que están disponibles.
- **Pueden manejar gran cantidad de datos.** Esto es debido a que utiliza una estructura distribuida, en muchos casos mediante tablas Hash.
- **No genera cuellos de botella.** El principal problema de los sistemas SQL es que necesitan transcribir cada sentencia para poder ser ejecutada, y cada sentencia compleja requiere además de un nivel de ejecución aún más complejo, lo que constituye un punto de entrada en común, que ante muchas peticiones puede ralentizar el sistema.

Algunas de las diferencias más destacables que nos podemos encontrar entre los SGBD NoSQL y los SGBD SQL están:

- **No utilizan SQL como lenguaje de consultas.** La mayoría de las bases de datos NoSQL evitan utilizar este tipo de lenguaje o lo utilizan como un lenguaje de apoyo. Por poner algunos ejemplos, Cassandra utiliza el lenguaje CQL, MongoDB utiliza JSON o BigTable hace uso de GQL.
- **No utilizan estructuras fijas como tablas para el almacenamiento de los datos.** Permiten hacer uso de otros tipos de modelos de almacenamiento de información como sistemas de clave-valor, objetos o grafos.
- **No suelen permitir operaciones JOIN.** Al disponer de un volumen de datos tan extremadamente grande suele resultar deseable evitar los JOIN. Esto se debe a que, cuando la operación no es la búsqueda de una clave, la sobrecarga puede llegar a ser muy costosa. Las soluciones más directas consisten en desnormalizar los datos, o bien realizar el JOIN mediante software, en la capa de aplicación.
- **Arquitectura distribuida.** Las bases de datos relacionales suelen estar centralizadas en una única máquina, en una estructura cliente-servidor, sin embargo en los casos NoSQL la información puede estar compartida en varias máquinas mediante mecanismos de tablas hash distribuidas.

### Tipos de bases de datos NoSQL

**(1) Clave-valor:** son el modelo de base de datos NoSQL más popular, además de ser la más sencilla en cuanto a funcionalidad. En este tipo de sistema, cada elemento está identificado por una llave única, lo que permite la recuperación de la información de forma muy rápida, información que habitualmente está almacenada como un objeto binario (BLOB). Se caracterizan por ser muy eficientes tanto para las lecturas como para las escrituras.



Las bases de datos clave-valor son altamente divisibles y permiten escalado horizontal a escalas que otros tipos de bases de datos no pueden alcanzar. Los casos de uso como juegos, tecnología publicitaria e IoT se prestan particularmente bien con el modelo de datos clave-valor.

De entre los sistemas de este tipo tenemos:

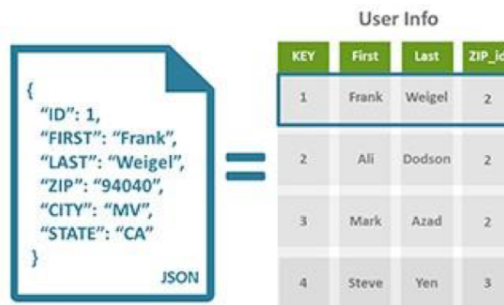
- Cassandra.
- Redis.
- DynamoDB.
- Memcached.
- Oracle NoSQL.
- Aerospike.
- Riak.

**(2) Documentos:** son otro tipo de base de datos NoSQL con un grado de complejidad y flexibilidad superior a las bases de datos clave-valor.

Este tipo almacena la información como un documento, generalmente utilizando para ello una estructura simple como JSON o XML y donde se utiliza una clave única para cada registro. Habitualmente, los documentos se estructuran dentro de una serie de contenedores llamados colecciones que son proporcionados por el sistema gestor documental.

Este tipo de implementación permite, además de realizar búsquedas por clave-valor, realizar consultas más avanzadas sobre el contenido del documento.

Son las bases de datos NoSQL más versátiles. Se pueden utilizar en gran cantidad de proyectos, incluyendo muchos que tradicionalmente funcionarían sobre bases de datos relacionales.



Algunos ejemplos de este tipo de SGBD son:

- MongoDB.
- CouchDB.
- SimpleDB.
- RethinkDB.

**(3) Grafos:** las bases de datos orientadas a grafos tienen la particularidad de representar la información como si de un grafo se tratara. La información viene representada por los nodos, y las relaciones entre los datos por las aristas. De este modo, se puede emplear la teoría de grafos para recorrer la base de datos y así gestionar y procesar la información

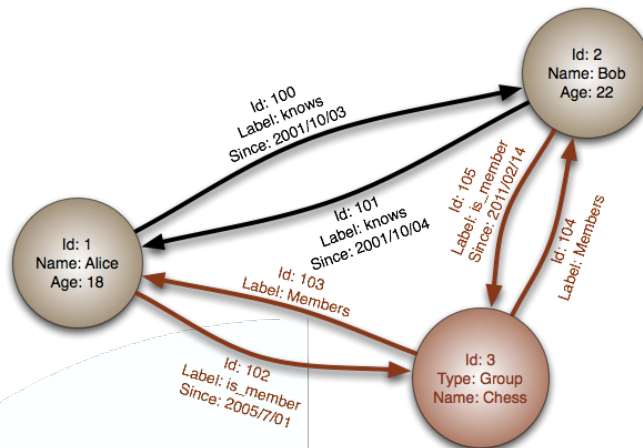
Una base de datos orientada a grafos, de forma generalizada, es cualquier sistema de información donde cada elemento tiene un puntero directo hacia sus elementos adyacentes, es decir, no sería necesario realizar consultas mediante índices.

A diferencia de la mayoría de bases de datos, el **rendimiento** de una base de datos orientada a grafos no se deteriora con el crecimiento de la base de datos, ni con consultas o procesamiento muy intensivos de los datos. Esto es debido a que el rendimiento será siempre proporcional al tamaño y el rendimiento que tenga el recorrer la parte del árbol que esté implicada, y no el tamaño global del grafo.

El modelo de grafos dota al desarrollador de aplicaciones de una elevada **flexibilidad** para manipular el conjunto de datos, ya que le permite conectar los datos de forma que posteriormente sea sencillo realizar consultas o actualizaciones desde una aplicación.

Este tipo de bases de datos nos permiten realizar un mantenimiento progresivo y **ágil** de los sistemas, a medida que la aplicación va creciendo, de forma que se puede adaptar a las nuevas necesidades del negocio.





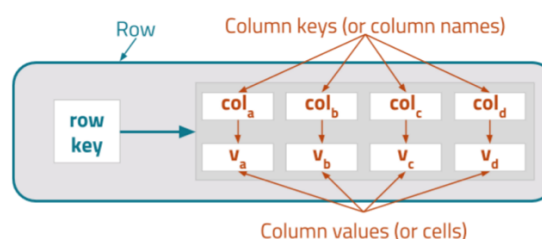
Las bases de datos orientadas a grafos más populares son:

- Neo4j.
- AllegroGraph.
- FlockDB.
- InfinityGraph.
- OrientDB (escrito en Java, es multimodelo).

**(4) Columnas o tabular:** las bases de datos orientadas a columnas son otro caso particular de la enorme familia de las bases de datos NoSQL. En este tipo de almacenes, en contraposición con el modelo relacional, la información se estructura en columnas en lugar de en filas.

Algunas de las bases de datos NoSQL más importantes y con una mayor aceptación pertenecen a este grupo. Bigtable, la solución NoSQL de Google, HBase, la base de datos de Hadoop, Cassandra, impulsada por Facebook y ahora bajo los brazos de Apache Software Foundation, son solo algunos ejemplos de este tipo de bases de datos.

Las comparativas de rendimiento entre los sistemas de gestión por filas (especialmente SGBDR) y los basados en columna, vienen principalmente de la mano de la eficiencia de los accesos que realizan a disco. Los accesos a posiciones consecutivas se producen en una cantidad de tiempo sensiblemente menor que los accesos que se producen a posiciones aleatorias a disco. Tener un sistema con multitud de accesos aleatorios a disco puede lastrar en gran medida el rendimiento del sistema de almacenamiento.



Ejemplos de bases de datos orientados a columnas:

- Bigtable.
- HBase.

Notar que Cassandra a veces es clasificada en este grupo también.

Algunos SGBD NoSQL son **multimodelo**, es decir, soportan varios modelos (por ejemplo, clave-valor, documentos y grafos). Destacan **OrientDB, ArangoDB o Azure Cosmos DB**.

Una vez enumeradas los SGBD NoSQL en función del tipo de sistema describiremos brevemente los más relevantes.

## 1. Cassandra (clave-valor)

Inicialmente desarrollada por Facebook, se trata de una base de datos de Apache del tipo clave-valor, altamente escalable, distribuida y eventualmente consistente. Asimismo, se considera un híbrido entre el modelo clave-valor y el modelo orientado a columnas o tabular.



Dispone de un lenguaje propio para realizar consultas **CQL** (Cassandra Query Language). Cassandra es una aplicación Java por lo que puede correr en cualquier plataforma que cuente con la JVM.

Actualmente Cassandra se utiliza en varios de los sitios más conocidos y con mayores necesidades en cuanto a tráfico y almacenamiento se refiere, como por ejemplo, Netflix, Ebay, Spotify, Twitter, Reddit, SoundCloud o Digg entre otros.

## 2. Redis (clave-valor)

Redis es un SGBD de tipo clave-valor **en memoria**, que además permite persistir los datos que residen en memoria a disco. Se puede imaginar como un array gigante en memoria para almacenar datos, datos que pueden ser cadenas, hashes, conjuntos de datos o listas.



El origen del nombre de la base de datos, Redis, es el acrónimo de **REmote DIctionary Server**. Redis funciona de forma similar a como lo hacen el resto de los SGBD clave-valor, presentando una serie de ventajas sobre éstas.

Creado en ANSI C y liberado bajo licencia BSD, es multiplataforma.

Redis es utilizado por algunas de las empresas más importantes del panorama actual de Internet, como Twitter, GitHub, Weibo, Digg, Hulu, StackOverflow, Instagram o Tumblr.

### 3. DynamoDB (clave-valor)

Amazon DynamoDB es un SGBD NoSQL del tipo clave-valor.



DynamoDB es un SGBD propietaria desarrollada internamente por Amazon y lanzada en 2007, que se incluye dentro de la oferta de servicios que se ofrecen como parte de los AWS (Amazon Web Services).

### 4. MongoDB (orientada a documentos)

Estamos ante el Sistema Gestor de Bases de Datos no relacionales (SGBD NoSQL) más **popular y utilizado** actualmente, bajo licencia GNU AGPL.



MongoDB es un SGBD NoSQL **orientado a documentos** que almacena la información en **estructuras BSON** (basado en JSON) con un esquema dinámico que permite su facilidad de integración.

Empresas como Google, Facebook, eBay, Cisco o Adobe utilizan MongoDB como Sistema Gestor de Bases de datos.

MongoDB está escrito en el lenguaje de programación C++.

Las principales **características** de MongoDB son:

- Indexación y replicación.
- Balanceo de carga.
- Almacenamiento en ficheros.
- Consultas ad hoc.
- Escalabilidad horizontal.
- Open Source.

Como **desventaja** principal, MongoDB no es un SGBD adecuado para realizar transacciones complejas.

### 5. CouchDB (orientada a documentos)

Se trata de un sistema orientado a documentos creado por Apache, escrito en lenguaje Erlang, multiplataforma, que almacena la información en estructuras JSON, garantizando la consistencia eventual y provee una semántica ACID.



Como características más importantes cabe destacar el uso de Restful HTTP API como interfaz y JavaScript como principal lenguaje de interacción. Para el almacenamiento de los datos se utiliza archivos JSON. Permite la creación de vistas, que son el mecanismo que permite la

combinación de documentos para retornar valores de varios documentos, es decir, CouchDB permite la realización de las operaciones JOIN típicas de SQL.

## 6. Neo4j (orientada a grafos)

Neo4j es una base de datos orientada a grafos, que está escrita en Java y desarrollada por la empresa Sueca Neo Technology. Neo4j, en la orientación a grafos, es la más conocida y utilizada de todas.



La principal característica es que está especialmente ideada para representar relaciones entre datos y datos conectados.

## 7. Bigtable (orientada a columnas)

Bigtable es un SGBD NoSQL orientado a columnas creado por Google. Como características destacan que es distribuido y la alta eficiencia que ofrece.



## 8. HBase (orientada a columnas)

HBase es un SGBD NoSQL orientado a columnas o tabular que forma parte del proyecto Hadoop de Apache. Escrito en Java, es de código abierto bajo licencia Apache License 2.0 y multiplataforma.



Se podría decir que HBase es la implementación Open Source del proyecto Bigtable de Google.

HBase es una base de datos distribuida, persistente, dispersa y ordenada en un mapa multidimensional. Una celda de esta base de datos viene definida por una clave primaria de fila (rowkey), por una clave de columna (columnkey) y una marca de tiempo (timestamp).

## 9. IndexedDB

Es una API de bajo nivel que ofrece almacenamiento persistente en el cliente, en concreto, en el navegador del usuario, de grandes cantidades de datos estructurados para administrar bases de datos NoSQL de objetos JSON. Permite trabajar de forma offline.

Su uso está pensado para aplicaciones que almacenan grandes cantidades de datos y para aplicaciones que no necesitan conexión permanente a Internet para funcionar.

IndexedDB permite almacenar y recuperar objetos que están indexados con una "clave". Todos los cambios que realiza en la base de datos ocurren dentro de las transacciones.

#### Características clave:

- Las bases de datos IndexedDB almacenan pares clave-valor. Los valores pueden ser objetos estructurados complejos y las claves pueden ser propiedades de esos objetos.
- IndexedDB se basa en un modelo de base de datos transaccional: todo sucede en el contexto de una transacción.
- La API IndexedDB es asíncrona (funcionamiento similar XMLHttpRequest).
- IndexedDB usa muchas solicitudes (objetos que reciben los eventos DOM).
- IndexedDB usa eventos DOM para notificarle cuando los resultados están disponibles.
- IndexedDB es orientado a objetos. No es una base de datos relacional con tablas que representan colecciones de filas y columnas.
- IndexedDB no utiliza lenguaje de consulta estructurado (SQL). Hace uso de consultas en un índice que produce un cursor, que se emplea para iterar en el conjunto de resultados.
- IndexedDB se adhiere a una política del mismo origen. Un origen es el dominio, el protocolo de la capa de aplicación y el puerto de una URL del documento donde se ejecuta el script. Cada origen tiene su propio conjunto asociado de bases de datos. Cada base de datos tiene un nombre que lo identifica dentro de un origen. El límite de seguridad impuesto a IndexedDB evita que las aplicaciones accedan a datos con un origen diferente. Por ejemplo, si bien una aplicación en `http://www.example.com/app/` puede recuperar datos de `http://www.example.com/dir/`, debido a que tienen el mismo origen, no puede recuperar datos de `http://www.example.com:8080/dir/` (puerto diferente) o `https://www.example.com/dir/` (protocolo diferente), porque tienen diferentes orígenes.

## 5. OTROS TIPOS DE BASES DE DATOS

Especial atención merecen las **bases de datos de series temporales**, las cuales recopilan, sintetizan y derivan información de manera eficiente a partir de datos que cambian con el tiempo y con consultas que abarcan intervalos de tiempo.

La aplicación práctica de este tipo de bases de datos es: análisis de información generada por sensores (IoT), flujos de clics, cotizaciones bursátiles o telemetría.

Destacan los siguientes productos:

- AWS Timestream.
- InfluxDB (código abierto).
- Prometheus.
- TimescaleDB: base de datos relacional con soporte para datos de serie temporal.
- Graphite.
- QuestDB.
- OpenTSDB.

